Project Title

[5]Enhancements for Flexbench: OpenAPI Integration and Anonymization using Machine Learning (ML)

Contributor Name: Yujun Liu

Email: yujunliu150@gmail.com

Github Username: AJun01

Location: Boston, MA, USA

Status: For Review

Project Abstract

HTTP request generator helps us simulate the scenarios that might happen after servers are deployed, helps developers to not only detect possible errors and perfect product before lunch but also helps maintaining and testing purposes. With the trending AI language model integration, HTTP testing apps like Flexbench could involved in handling more and achieving better potentials, such as creating realistic scenarios: AI can analyze patterns in real traffic and create simulated requests that mimic these patterns. This can be used for load testing or to train the application to handle different types of requests effectively. By engineering the prompt and responsive APIs, we will be able to simulate more realistic. What's more, automated testing: AI can automate the process of generating HTTP requests for testing endpoints, reducing the need for manual testing and thereby increasing efficiency. By configuring numbers or prompts at the app's interface, AI will automatically modify incoming fake HTTP requests and simulate non-stop.

Flexbench serves a practical role in generating simulated HTTP traffic for load testing, and the proposed enhancements aim to elevate its utility. My approach leverages OpenAI API & Developer Platform, prompt engineering, and Full-Stack engineering support to streamline test scenario generation. My strategy encompasses algorithm development that handles strings for efficient traffic simulation and employs existing machine-learning models with designed APIs for data anonymization. The inclusion of a responsive UI and thorough documentation will be paramount.

Project Proposal

Goals:

- **GPT API Integration**: Develop APIs to handle GPT responses for generating test scenarios.
- **Data Anonymization Module**: Create a module to anonymize sensitive data using prompt engineering with GPT.
- **Deployment and Testing**: Integrate these features into Flexbench's codebase and ensure full functionality through testing.
- **UI/UX desktop-app fix**: Enhance the user interface and experience to ensure the Flexbench desktop app is intuitive and accessible for all users.
- **Documenting and Debugging:** Create and maintain detailed documentation for all new implementations and establish a robust debugging framework to identify and resolve issues promptly.

Detailed Proposal(High-Level Overview:):

Front-end Design(desktop-app):

App page will be divided up into two major sections: the input field & submit button

1. **URL Input field**: The input format will require the user to input data for each section of an HTTP request.
2. **Method selecto**r: selector allows users to select from GET/POST/DELETE/PUT
3. **Body Section**: A text area for the body of the request, which could include tabs for different content types (e.g., raw JSON, form data).
4. **Customized Prompt field**: a text area for the extra prompt that users want to add to the GPT HTTP request generation
5. **Response field**: a text area for the extra prompt that users want to add to the GPT HTTP request generation, for example :

   Host: 'www.example.com' Port: '80';

   Path : '/'; Method : 'GET';

   Body: 'Hello world';

   Customized Prompt: 'What else do you wish you add for HTTP requests'

6. **Button(Submit)**: After clicking the Submit button will trigger the event onSubmit pass down the data to OpenAI API

Back-end Design:

1. **Sign Up for OpenAI and Obtain the API Key:**

   To follow up with the instructions on OpenAI development, we need to sign up for an OpenAI account(Google || Apple || Microsoft).

2. **Set Up Lib for Flexbench(include all related tools):**

   In the development environment of Flexbench(Docker, VScode, can install OpenAI library: `npm install --save openai`

   Also, we will be using the Axios package to make HTTP requests to the OpenAI API: `% npm install axios`

3. **Write the Code to Make API Calls:**

   I will need to write JavaScript functions that use Axios to send POST requests to the OpenAI API endpoints, passing in the API key for authentication and the prompt for the GPT model to generate the response.

   a. Design Flexbench API Interface handles onSubmit event (/generate-text => {})

      Create function /generate-text => {} to collect data from the front-end event triggered. returns the full prompt sentences that are ready to POST.

   b. Set Up a Server to Host the API(Node.js, Express)

      Craft a server that assists the API manage data flows using Express, it is possible to use architectures like microservices to handle different features for setting up a good robust frame for future contributors

   c. Implement the OpenAI API Call

      Use Axios to make HTTP requests to the OpenAI API within the API's code. What’s more, it Handles POST from the generate-text function that gathers data from the **desktop app end and redirects it to GPT API endpoints.** Configure the Express.js server to parse incoming data in **JSON** requests using express.json().

      While handling the data, it comes the technique for Validating and anonymization.

d. Validate Incoming Data(trim, format)

Add validation logic to check the correctness and completeness of the incoming requests to your API. If the generated prompt violates a certain format, will respond to the response field with an error message

e. Anonymization:

Streamline incoming data validation to ensure accuracy. Integrate an anonymization step to **remove any personal details from data** before API processing. This protects privacy and complies with data standards.

f. **Summary(**Simplified dataflow**):**

Designed API received data from Flexbench desktop-app simulation of HTTP generation button event

API handles the event:

API got it, formatted it and rerouted it(from the client).

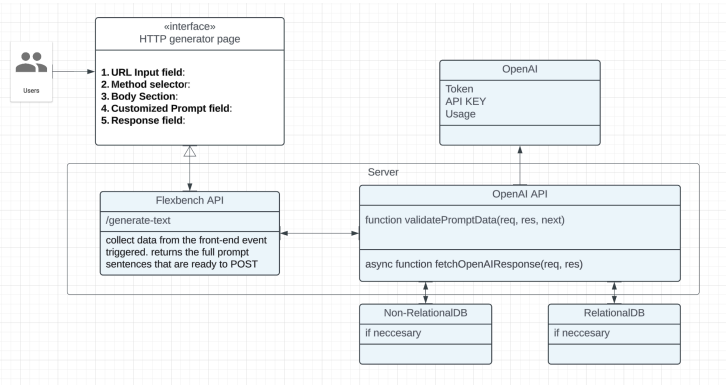
Make HTTP Requests to OpenAI API & Process OpenAI API Responses. Parse the response from OpenAI and format it as needed before sending it back to the client software.

API got it, formatted it, rerouted it(from GPT).

Send Back the Response to the desktop app interface for showcasing the data(simulated HTTP requests from GPT)

4. **Testing:** Thoroughly test the implementation to ensure it correctly interacts with the OpenAI API and handles responses properly.

UML:



Pre-Proposal Works

Familiarization: I will dive deep into the existing Flexbench documentation and codebase to understand the current setup and functionalities. Understanding the current state of the project is critical to ensuring that new features integrate seamlessly.

Research: I will conduct extensive research on best practices for API integration and data anonymization. This will include reviewing academic literature, existing open-source projects, and the latest OpenAI API documentation to stay up-to-date with the most recent developments and capabilities.

Prototype Development: Early-stage development of prototypes for the OpenAPI integration and data anonymization module will begin. These prototypes will be designed to test the feasibility of the proposed features and to identify any potential roadblocks early on.

Engagement with the Community: I will reach out to the Flexbench user community to gather feedback on desired features and pain points with the current system. This will help in prioritizing the work during the project execution phase. The most importantly, communicate with mentor about ideas and practical implementation ensure feasibility.

Source:

React - https://www.udemy.com/course/microservices-with-node-js-and-react/

Open AI -https://platform.openai.com/docs/overview

Axios - https://www.youtube.com/watch?v=6LyagkoRWYA

HTTP - https://www.youtube.com/watch?v=iYM2zFP3Zn0

Full-Stack Web Development: https://www.youtube.com/watch?v=nu_pCVPKzTk

Contributor Background

Education

- Boston University (Master's degree)- Computer Science

- Pennsylvania State University (Bachelor Degree) - Information Science and Technology

Project Experience

SmartSort AI-Integrated File Sorter with OpenAI-API

- Developed 'SmartSort', an AI-driven file organization tool leveraging machine learning to automate the categorization and sorting of files across a user's system. Spearheaded the integration of OpenAI's API to analyze file content, optimizing file management tasks by implementing custom sorting algorithms.

AI-NPC TalkBot Mini Game with Unity Engine & OpenAI-API

- Developed a unique game feature that utilizes OpenAI's ChatGPT for real-time interaction with AI-driven non-player characters (NPCs) by interfacing with a web-based API and local text documents.Enhanced the gaming experience by enabling dynamic conversations with AI NPCs, showcasing the potential for innovative AI applications in gaming despite practical limitations.

Full-Stack: React Web App Based On Micro Services | *Javascript, Node.js, Axios, Nodemon*

- *Created a blog-sharing web app that allowed users to post and comment on conversations on the web community*
- Designed and implemented a full stack solution for real-time blog sharing, boasting over **35 registered** users
- Built highly extensible and robust back-end by implementing **React** with microservices architecture using **Express.**
- Implemented a solution that optimized the response time of **comment-sensor** processing via a crafted moderation service paired with a query service, enabling comment filtering with **minimal latency** through an **event bus**.

Professional Experience

- Full Stack Software Engineer Intern (Sep. 2021 - Jan. 2022)
- Technical Manager Trainee (Jan. 2023 - July. 2023)

Objective:

As a computer science student with a focus on full-stack Web/Software Engineering and up-to-date AI trending technologies, I have garnered extensive work and personal experiences in **NodeJS,** machine learning, and **GPT API integration** with **very strong interests**(the future is here!). My skills are directly applicable to the Flexbench project. My history includes the successful deployment of **AI-driven applications and microservices with React**, which aligns with Flexbench's development for the Google Summer of Code this year.

The synergy between my hands-on experiences with JavaScript, React, and backend development, along with my academic pursuits and my AI fascination, positions me uniquely to contribute effectively to this project. I was **very excited** when I came across this project and of course, I am **eager to apply** my skills to advance Flexbench while enriching the open-source community with robust, innovative testing solutions.

Supportive:

- Resume
- LinkedIn
- Github

Related Works

Several related works address similar problems in web development environments and tools with OpenAI API.

1. **JavaScript and OpenAI API Guide:** This resource provides a detailed explanation of how to use the OpenAI API with JavaScript. It is a primer for developers interested in integrating AI capabilities into their applications, especially focusing on text translation tasks using the latest GPT-3.5-Turbo and GPT-4 models. (evolvingDev).
2. **OpenAI ChatGPT API With React JS:** Aimed at web developers, this comprehensive guide explores the integration of OpenAI's ChatGPT API with React JS applications. It outlines the process from obtaining an API key to creating a user-friendly chat interface. (DZone).
3. **OpenAI API in NextJS 14:** This step-by-step guide is perfect for developers working with Next.js. It explains integrating OpenAI's powerful API into Next.js applications, enabling enhanced features like natural language processing and content generation. (Satria AI - AI for Business Growth).

Project Schedule(Full-time)

- May 1 – May 27 (Before the official coding time)

  Understand the current Flexbench system, focusing on areas where GPT-3.5 integration could be beneficial.
  Agile develops an API that interacts with GPT-3 to generate test scenarios and anonymize data.

  - Week1: Read OpenAI API Documents.
  - Week2: Design the API UML&Dataflow
  - Week3: Design prompt and testing
  - Week4: Read Flexbench documents and codebase

- May 27 – June 28 (Official coding period starts):

  - Week5: Implement designed API endpoints in Flexbench's codebase
  - Week6: Implement designed prompt
  - Week7: Build functionality to parse the response from OpenAPI
  - Week8: Map responses to Flexbench scenarios

- JULY 8th MID TERM EVALUATION
  Presenting and discussing ideas and revising works

- July 12 – August 19:

  - Week9: API Functionality integration with Flexbench codebase
  - Week10: Unit Testing
  - Week11: Git commit to GSOC branches with versioning
  - Week12: Update Documentation & Final Preparations

- August 19 – August 26:
  Mentors submit final contributor evaluations

- August 26 – September 3 :
  Final results posted

Planned Work Hours

TimeZone: GMT-4 EDT (Boston, MA)

Work Hours:

Monday - Thursday (8 hours per day)

- 9:00 a.m. - 11:30 a.m.
- 1:00 p.m. - 4:00 p.m.
- 9:00 p.m. - 9:30 p.m.

Friday (8 hours per day)

- 9:00 a.m. - 11:30 a.m.
- 1:00 p.m. - 6:30 p.m.

Planned Absence/Vacation Days

Final exams:

MET	CS566	Analys of Algrt	Wed	05/08/2024	06:00PM - 08:00PM
MET	CS575	Oprtng Systems	Tue	05/07/2024	06:00PM - 08:00PM
MET	CS662	Comp Lang Theor	Mon	05/06/2024	06:00PM - 08:00PM
MET	CS665	Sw Design & Pat	Thu	05/09/2024	06:00PM - 08:00PM

May 27 – August 19: Weekends(Saturday & Sunday)